ANALYTIXAI · SQL PRACTICE PACK 01

Who are our top 10 customers?

The exact dataset from the reel, plus five questions a real manager would ask. Practise grouping in SQL, and learn to use Claude to understand every line instead of just copying it.

1,240	1,885	₹57,14,000	Mar–Aug
customers	orders	total revenue	2026, six months

INSIDE

01 The dataset: two tables, one story

02 Set it up in two minutes

03 Warm-up: recreate the reel

04 Five practice questions

05 Answer key with what to tell your manager

HOW TO USE THIS PACK

1. **Load the data** into MySQL Workbench (page 2).
2. **Try each question yourself first.** Stuck? Use the Claude prompt on the card, and ask it to explain, not just answer.
3. **Check the answer key**, then compare your query with it line by line.

> Same data as the reel: run the video's query and you get the same top 10 and the same 32%.

SKILLS YOU WILL PRACTISE

`GROUP BY` `JOIN` `SUM · COUNT` `COUNT(DISTINCT)` `HAVING` `DATE_FORMAT` `LEFT JOIN` `Prompting Claude`

01 · THE DATASET

Two tables, one story

`sales_db` holds an online store's sales from 2 Mar 2026 to 30 Aug 2026. Every order belongs to exactly one customer.

customers — 1,240 rows

`customer_id` INT · PRIMARY KEY	Unique ID for each customer
`name` VARCHAR(60)	Full name. **Not guaranteed unique.**
`city` VARCHAR(40)	Where the customer is based (10 cities)

orders — 1,885 rows

`order_id` INT · PRIMARY KEY	Unique ID for each order
`customer_id` INT · FOREIGN KEY	Who placed it, links to customers
`order_date` DATE	When it was placed
`amount` DECIMAL(10,2)	Order value in rupees (₹)

`orders.customer_id → customers.customer_id` One customer can have many orders. That is why grouping matters.

FIRST 5 ORDERS

ORDER_ID	CUSTOMER_ID	ORDER_DATE	AMOUNT
10418	214	2026-03-02	₹2,450
10419	87	2026-03-02	₹12,990
10420	5	2026-03-03	₹860
10421	214	2026-03-03	₹5,340
10422	131	2026-03-04	₹1,725

FIRST 5 CUSTOMERS

CUSTOMER_ID	NAME	CITY
1	Sneha Agarwal	Delhi
2	Kabir Kapoor	Jaipur
3	Reyansh Shah	Bengaluru
4	Yash Bose	Pune
5	Shruti Menon	Bengaluru

02 · SET IT UP IN TWO MINUTES

MYSQL WORKBENCH (RECOMMENDED)

1. Open MySQL Workbench and connect to your local server.
2. **File → Open SQL Script...** and choose `sales_db.sql`.
3. Click the **lightning bolt** to run the whole script.
4. In the Navigator, right-click **Schemas → Refresh All**. `sales_db` appears with both tables.

WHERE ARE THE FILES?

Three files come with this pack: `sales_db.sql` (everything in one script), `customers.csv` and `orders.csv`.

They are also **attached inside this PDF**: open the attachments (paperclip) panel in your PDF reader.

Works on MySQL 8 and MariaDB. Using another database? Ask Claude to translate `DATE_FORMAT` in Q1.

03 · WARM-UP

Recreate the reel

Run the same query as the video. If your result matches, your setup is perfect.

```sql
SELECT c.name, SUM(o.amount) AS total_spent
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC
LIMIT 10;
```

YOU SHOULD SEE

NAME	TOTAL_SPENT
Arjun Mehta	₹2,46,500
Kavya Reddy	₹2,18,300
Rohan Gupta	₹1,97,800
Ananya Iyer	₹1,86,400
Vikram Singh	₹1,79,900
Meera Joshi	₹1,71,250
Aditya Rao	₹1,64,700
Sara Thomas	₹1,58,600
Karan Malhotra	₹1,55,300
Nisha Pillai	₹1,49,850

```sql
SELECT SUM(amount) AS total_revenue
FROM orders;
```

THE INSIGHT FROM THE REEL

Total revenue is **₹57,14,000.** The top 10 customers spent **₹18,28,600** of it.

32%

of all revenue comes from just 10 of 1,240 customers.

Why `GROUP BY c.customer_id, c.name` **?** Two customers can share a name. Grouping by the ID keeps them apart. Question 4 shows what happens if you forget.

WHAT EACH LINE DOES (THE SAME EXPLANATION AS THE REEL)

`JOIN customers c ON …`	Connects each order to its customer, so names sit next to amounts.
`GROUP BY c.customer_id, c.name`	Puts each customer's orders together: many orders become one row per customer.
`SUM(o.amount)`	Adds up the orders inside each group.
`ORDER BY total_spent DESC`	Biggest spenders first.
`LIMIT 10`	Keeps only the top ten.

04 · PRACTICE QUESTIONS

Five questions a manager would really ask

Try each one before you look at the answer key. Every question uses grouping, the one skill from the reel.

Q1 GROUP BY WITH DATES — LEVEL ●○○

Are we growing month by month?

YOUR MANAGER ASKS

“Can you show me revenue for each month since March? I want to know if we are growing.”

YOUR TASK Return one row per month with **month**, **number of orders** and **revenue**, oldest month first.

HINT Group by the month part of `order_date`. In MySQL, `DATE_FORMAT(order_date, '%Y-%m')` turns 2026-03-14 into 2026-03.

ASK CLAUDE THE RIGHT WAY

```
I'm using MySQL. Tables: customers(customer_id, name, city), orders(order_id,
customer_id, order_date, amount). Write a query that shows the number of orders and
total revenue for each month, oldest first. Explain each line like I'm a beginner.
```

Q2 JOIN + GROUP BY — LEVEL ●●○

Which city should we focus on?

YOUR MANAGER ASKS

“Which city brings in the most revenue? Show me how many customers and orders each city has too.”

YOUR TASK Return one row per city with **city**, **customers** (who ordered), **orders** and **revenue**, highest revenue first.

HINT `city` lives in customers and `amount` lives in orders, so you need a JOIN. Count customers with `COUNT(DISTINCT ...)`, otherwise you count orders twice.

ASK CLAUDE THE RIGHT WAY

```
I'm using MySQL. Tables: customers(customer_id, name, city), orders(order_id,
customer_id, order_date, amount). For each city, show the number of customers who
ordered, the number of orders and total revenue, highest revenue first. Explain why
COUNT(DISTINCT) is needed.
```

Q3 GROUP BY + HAVING

LEVEL ●●○

Do our customers come back?

YOUR MANAGER ASKS

“How many customers ordered only once and never came back?”

YOUR TASK Return **one number**: how many customers have exactly one order. **Bonus:** list the 5 biggest one-time buyers (name, city, amount). They are worth a personal follow-up.

HINT `WHERE` filters rows *before* grouping, `HAVING` filters groups *after*. Group orders by customer, then keep the groups where `COUNT(*) = 1`.

ASK CLAUDE THE RIGHT WAY

```
I'm using MySQL. Tables: customers(customer_id, name, city), orders(order_id,
customer_id, order_date, amount). Find how many customers have exactly one order,
then list the 5 biggest one-time buyers. Explain the difference between WHERE and
HAVING.
```

Q4 THE NAME TRAP

LEVEL ●●●

Why is my top 10 different from yours?

YOUR MANAGER ASKS

“A colleague sent me a top-10 customer list that doesn't match yours. Whose is right?”

YOUR TASK 1) Find every **name shared by more than one customer**. 2) Run the top-10 query grouping by **name only** and compare it with the reel. What changed, and why?

HINT `GROUP BY name HAVING COUNT(*) > 1` finds repeated names. For part 2, group by `c.name` instead of `c.customer_id, c.name`.

ASK CLAUDE THE RIGHT WAY

```
I'm using MySQL. Tables: customers(customer_id, name, city), orders(order_id,
customer_id, order_date, amount). Show me names that belong to more than one
customer_id. Then explain what goes wrong if I GROUP BY name instead of customer_id
in a top-customers query.
```

Q5 LEFT JOIN

LEVEL ●●●

Who never bought anything?

YOUR MANAGER ASKS

“Marketing wants everyone who signed up but never placed an order, so they can send a welcome offer.”

YOUR TASK Return **customer_id**, **name** and **city** for every customer with zero orders. How many are there?

HINT An inner `JOIN` silently drops customers with no orders. `LEFT JOIN` keeps every customer; the order columns come back `NULL` for people who never ordered.

ASK CLAUDE THE RIGHT WAY

```
I'm using MySQL. Tables: customers(customer_id, name, city), orders(order_id,
customer_id, order_date, amount). List the customers who have never placed an order.
Explain why a LEFT JOIN is needed and what WHERE o.order_id IS NULL does.
```

BEFORE YOU OPEN THE ANSWER KEY

1. **Read your result like a manager.** Does the number make sense? Would you bet your review on it?
2. **Ask Claude to review your query**, not write it: “Here is my query and its result. Is it correct? What could go wrong?”
3. **Write one sentence** you would send your manager. The answer key has one for every question.

Q1 Are we growing month by month?

```sql
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       COUNT(*)    AS orders,
       SUM(amount) AS revenue
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

MONTH	ORDERS	REVENUE
2026-03	211	₹6,43,565
2026-04	261	₹7,75,820
2026-05	281	₹8,78,550
2026-06	326	₹10,08,115
2026-07	375	₹11,86,770
2026-08	431	₹12,21,180

TELL YOUR MANAGER

Revenue grew every month, from ₹6,43,565 in March to ₹12,21,180 in August: up 90% in six months.

Q2 Which city should we focus on?

```sql
SELECT c.city,
       COUNT(DISTINCT c.customer_id) AS customers,
       COUNT(o.order_id)             AS orders,
       SUM(o.amount)                 AS revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.city
ORDER BY revenue DESC;
```

CITY	CUSTOMERS	ORDERS	REVENUE
Delhi	**186**	**299**	**₹9,62,420**
Mumbai	207	314	₹9,35,070
Bengaluru	175	257	₹7,75,680
Hyderabad	133	227	₹6,69,730
Chennai	104	170	₹5,55,760
Pune	96	157	₹4,63,090
Kochi	**41**	**112**	**₹3,92,100**
Kolkata	95	127	₹3,50,040
Jaipur	62	116	₹3,41,580
Ahmedabad	83	106	₹2,68,530

TELL YOUR MANAGER

Delhi leads with ₹9,62,420, just ahead of Mumbai (₹9,35,070), even though Mumbai has more customers (207 vs 186). Kochi punches above its weight: 41 customers placed 112 orders.

Q3 Do our customers come back?

```sql
-- customers who ordered exactly once
SELECT COUNT(*) AS one_time_buyers
FROM (
  SELECT customer_id
  FROM orders
  GROUP BY customer_id
  HAVING COUNT(*) = 1
) AS t;
```

ONE_TIME_BUYERS
1034

```sql
-- bonus: the 5 biggest one-time buyers
SELECT c.name, c.city,
       SUM(o.amount) AS spent
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name, c.city
HAVING COUNT(o.order_id) = 1
ORDER BY spent DESC
LIMIT 5;
```

NAME	CITY	SPENT
Siddharth Shah	Bengaluru	₹9,690
Ananya Nair	Delhi	₹9,470
Anjali Chopra	Bengaluru	₹7,260
Shruti Bhatt	Delhi	₹7,240
Vivaan Desai	Chennai	₹7,150

TELL YOUR MANAGER

1,034 of our 1,182 paying customers (87%) ordered once and never came back. Repeat business is the biggest opportunity.

Q4 Why is my top 10 different from yours?

```sql
-- 1) names used by 2+ customers
SELECT name, COUNT(*) AS people
FROM customers
GROUP BY name
HAVING COUNT(*) > 1;
```

NAME	PEOPLE
Amit Kumar	2
Priya Sharma	2
Rohan Gupta	2

```sql
-- 2) the trap: group by name only
SELECT c.name,
       SUM(o.amount) AS total_spent
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
GROUP BY c.name
ORDER BY total_spent DESC
LIMIT 10;
```

Q4 The result, side by side

GROUPED BY NAME (WRONG)

NAME	TOTAL_SPENT
Arjun Mehta	₹2,46,500
Rohan Gupta	**₹2,21,200**
Kavya Reddy	₹2,18,300
Ananya Iyer	₹1,86,400
Priya Sharma	**₹1,82,800**
Vikram Singh	₹1,79,900
Meera Joshi	₹1,71,250
Aditya Rao	₹1,64,700
Sara Thomas	₹1,58,600
Karan Malhotra	₹1,55,300

GROUPED BY ID, AS IN THE REEL (RIGHT)

NAME	TOTAL_SPENT
Arjun Mehta	₹2,46,500
Kavya Reddy	₹2,18,300
Rohan Gupta	₹1,97,800
Ananya Iyer	₹1,86,400
Vikram Singh	₹1,79,900
Meera Joshi	₹1,71,250
Aditya Rao	₹1,64,700
Sara Thomas	₹1,58,600
Karan Malhotra	₹1,55,300
Nisha Pillai	**₹1,49,850**

TELL YOUR MANAGER

3 names belong to two different people each (Amit Kumar, Priya Sharma, Rohan Gupta). Grouping by name merges them: Rohan Gupta jumps to #2, a 'Priya Sharma' who is really two people appears at #5, and Nisha Pillai (a real top-10 customer) drops off the list. Always group by the ID.

Q5 Who never bought anything?

```sql
SELECT c.customer_id, c.name, c.city
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL
ORDER BY c.customer_id;
```

CUSTOMER_ID	NAME	CITY
1	Sneha Agarwal	Delhi
36	Nikhil Bhatt	Pune
61	Suresh Chauhan	Mumbai
65	Ishaan Patel	Mumbai
86	Vihaan Banerjee	Delhi

...and 53 more (58 customers in total)

TELL YOUR MANAGER

58 customers signed up but never placed an order: a ready-made list for a welcome offer.

Solved all five?

Comment the question you found hardest on the reel, and follow **analytixai_** on Instagram for more such content.